

WFLOW-02: Triggers & Event Types

Series: WFLOW — Workflows and Alert Notifications | **Notebook:** 2 of 10 | **Created:** January 2026 | **Last Updated:** 07/31/2026

Event-Driven Workflow Triggers

Triggers determine when workflows execute. This notebook covers all trigger types, detected problem events, metric events, schedules, and custom event triggers.

Table of Contents

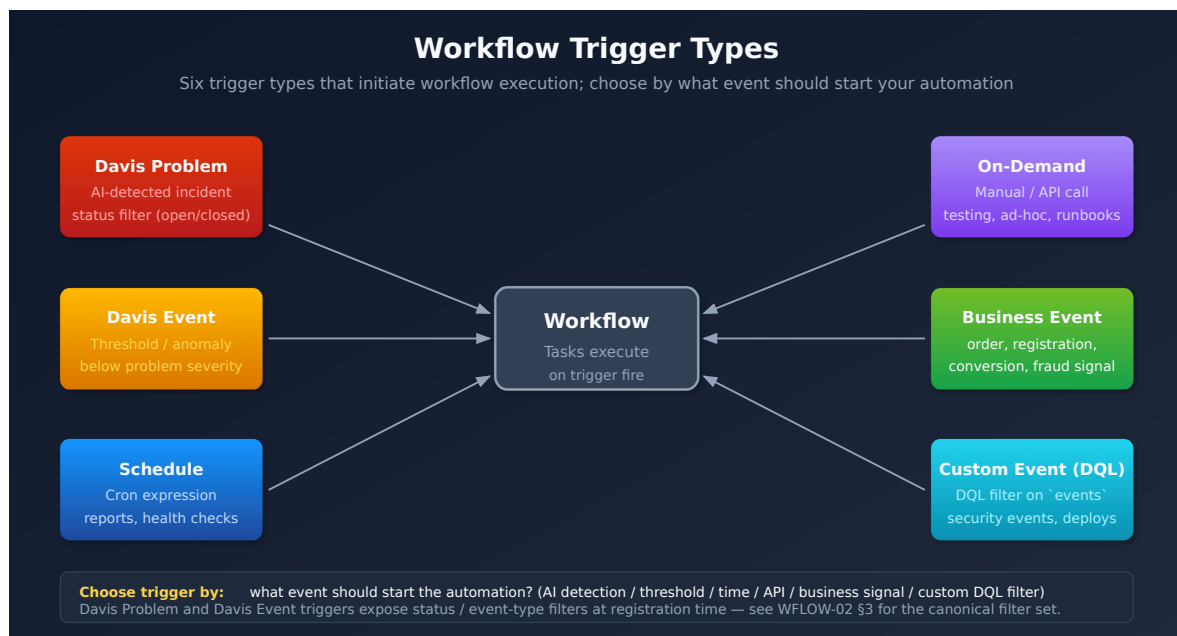
- 1. [Detected Problem Trigger](#)
- 2. [Detected Event Trigger \(Metrics\)](#)
- 3. [Schedule Trigger](#)
- 4. [On-Demand Trigger](#)
- 5. [Event Trigger \(Custom/Business Events\)](#)
- 6. [Trigger Data and Expressions](#)
- 7. [Davis Problem Event Payload Reference](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	<code>automation:workflows:write</code>
Prior Knowledge	WFLOW-01: Workflow Fundamentals

1. Trigger Types Overview

Trigger Type	Fires When	Primary Use Case
Detected Problem	Dynatrace Intelligence detects/updates/closes a problem	Alert notifications, incident management
Detected Event	Metric threshold breached	Capacity alerts, proactive notifications
Schedule	Cron expression matches	Reports, health checks, cleanup jobs
On-Demand	Manual execution or API call	Testing, ad-hoc automation
Event	Business/custom event ingested	Business process automation



Choosing the Right Trigger

Scenario	Recommended Trigger
"Notify when a service is slow"	Detected Problem
"Alert when CPU > 90% for 5 mins"	Detected Event
"Send weekly status report"	Schedule
"Process order completion events"	Event (bizevents)

Scenario	Recommended Trigger
"Test my workflow"	On-Demand

2. Detected Problem Trigger

The most common trigger for alert notifications. Fires when Dynatrace Intelligence detects problems.

Trigger Configuration

Setting	Description	Example
Problem state	Active only, or active + closed	Active
Categories	Problem categories to include	Infrastructure, Application
Severity	Minimum severity threshold	CRITICAL
Affected entities — tags	Tags the problem's affected entities must carry. Three modes: include all / all defined tags / any defined tag	team:checkout
Additional custom filter query	A DQL matcher on the incoming problem record (a subset of DQL — no aggregation, no querying across a set of events)	<code>maintenance.is_under_maintenance == false</code>
Updates	Re-trigger when selected fields change	Root cause changed

 **There is no Management Zone filter on the problem trigger.** An earlier revision of this notebook listed one; that was wrong. Dynatrace's alert-notification upgrade guide states the Management Zone filter is *"no longer supported — use Grail record-based field filters instead."* Scope the trigger with **affected-entity tags** plus the custom DQL matcher instead.

This matters most if you are migrating off Management Zones: a workflow whose condition reads `event()[ "management_zones" ]` keeps evaluating after the zones are

deleted, but against an **empty array** — so every condition silently goes false and the workflow stops routing without erroring. See MZ2POL-01 §5 for the full MZ-job-to-successor mapping.

Validate the filter before you save it. The trigger configuration offers **Query past events**, which estimates how many matching events occurred in your environment over recent windows. Use it on every non-trivial trigger.

The result to distrust is **zero across every window in an environment you know is busy**. A trigger can be syntactically valid, save without error, and still never fire — nothing surfaces an error to tell you, because an over-constrained filter is not a malformed one. Check in particular that the **categories selected above and any category named in the custom filter query agree**: a matcher narrowing to a category you did not select leaves nothing that can ever match. Reading the estimate at authoring time costs seconds and is the only signal you get.

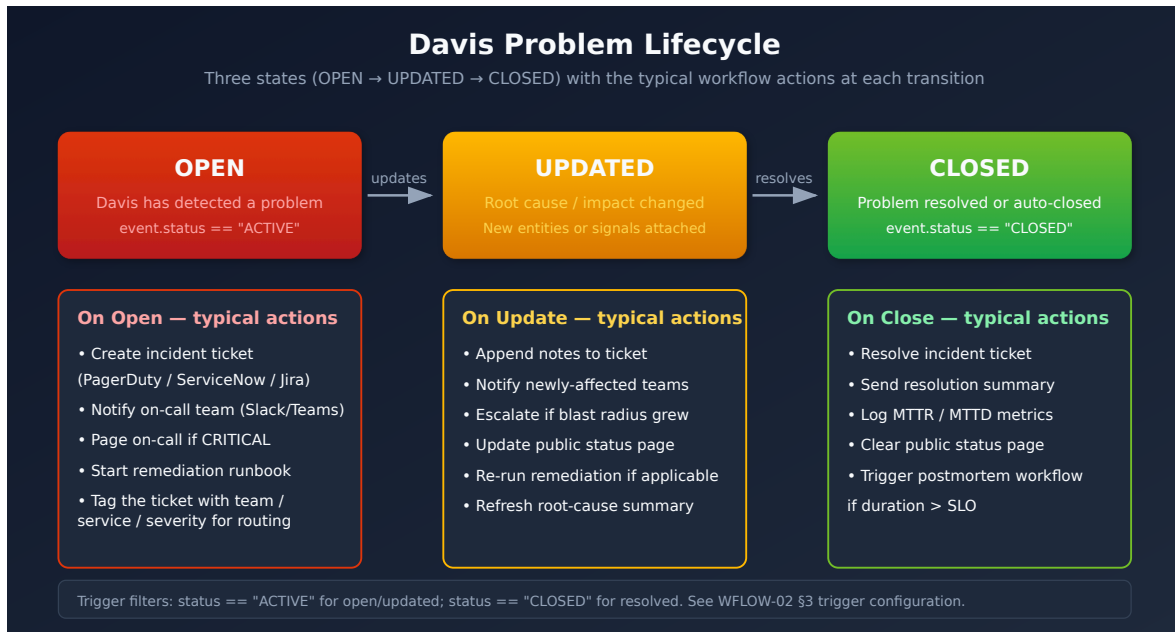
Problem Event Data

When a detected problem triggers, you get access to:

```
{
  "display_id": "P-12345",
  "title": "High response time on checkout service",
  "severity": "CRITICAL",
  "status": "OPEN",
  "start_time": "2026-01-27T10:00:00Z",
  "affected_entity_ids": ["SERVICE-ABC123"],
  "root_cause_entity_id": "HOST-XYZ789",
  "management_zones": ["Production"],
  "impacted_entities": [...],
  "problem_url": "https://env.dynatrace.com/ui/problems/P-12345"
}
```

Problem Lifecycle Events

Event	When	Typical Action
Problem opened	New problem detected	Create incident, notify team
Problem updated	Root cause/impact changed	Update incident notes
Problem closed	Problem resolved	Close incident, send summary



Example: Filter Critical Production Problems

```
trigger:
  type: davis-problem
  config:
    categories:
      - AVAILABILITY
      - PERFORMANCE
    entityTagsMatch: all
    entityTags:
      - key: env
        value: prod
```

3. Detected Event Trigger (Metrics)

Trigger based on metric thresholds without creating a detected problem.

When to Use

- **Proactive alerts** before Dynatrace Intelligence detects a problem
- **Capacity warnings** (disk 80%, memory usage)
- **Business KPIs** (orders/minute, cart abandonment)
- **Custom thresholds** different from baselines

Configuration

Setting	Description
DQL Query	Metric query defining the threshold
Evaluation Frequency	How often to check (1m - 1h)
Alert Condition	When the threshold is breached

Example: CPU Warning at 80%

```
trigger:
  type: davis-event
  config:
    query: |
      timeseries avg_cpu = avg(builtin:host.cpu.usage), by:{dt.entity.host}
      | filter avg_cpu > 80
    evaluationFrequency: "5m"
```

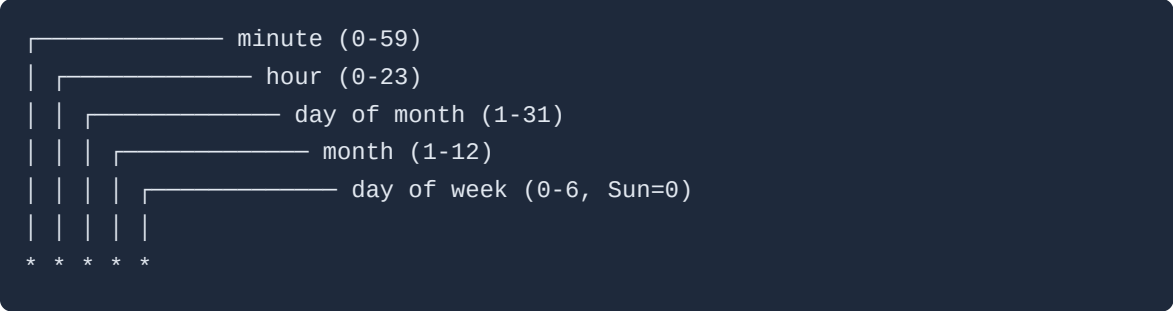
Event Data

```
{
  "metric_key": "builtin:host.cpu.usage",
  "value": 85.5,
  "threshold": 80,
  "entity_id": "HOST-ABC123",
  "triggered_at": "2026-01-27T10:00:00Z"
}
```

4. Schedule Trigger

Execute workflows on a recurring schedule using cron expressions.

Cron Expression Format



Common Schedules

Schedule	Cron Expression	Description
Every hour	<code>0 * * * *</code>	Top of every hour
Daily at 9 AM	<code>0 9 * * *</code>	Every day at 9:00
Weekdays at 8 AM	<code>0 8 * * 1-5</code>	Mon-Fri at 8:00
First of month	<code>0 0 1 * *</code>	1st day, midnight
Every 15 minutes	<code>*/15 * * * *</code>	0, 15, 30, 45 past

Example: Daily Health Check Report

```
trigger:
  type: schedule
  config:
    cron: "0 9 * * 1-5" # Weekdays at 9 AM
    timezone: "America/New_York"
```

Schedule Best Practices

- **Avoid minute 0** - Many workflows run at :00, spread load
- **Use timezones** - Be explicit about timezone
- **Consider rate limits** - Don't schedule too frequently

5. On-Demand Trigger

Manual execution for testing, ad-hoc runs, or API-triggered automation.

Manual Execution

1. Open workflow in editor
2. Click **Run** button
3. Optionally provide input parameters
4. View execution results

API Execution

Trigger workflows programmatically via API:

```
curl -X POST
"https://<env>/platform/automation/v1/workflows/<id>/run" \
-H "Authorization: Api-Token <token>" \
-H "Content-Type: application/json" \
-d '{"params": {"custom_param": "value"}}'
```

Input Parameters

Define custom parameters for on-demand workflows:

```
trigger:
  type: on-demand
  config:
    parameters:
      - name: environment
        type: string
        required: true
      - name: notify_slack
        type: boolean
        default: true
```

Access in tasks:

```
{{ trigger().params.environment }}
{{ trigger().params.notify_slack }}
```


6. Event Trigger (Custom/Business Events)

Trigger on business events or custom events ingested into Grail.

Use Cases

- **Order completed** → Update inventory system
- **User registered** → Send welcome notification
- **Deployment finished** → Run validation tests
- **Custom alert** → External system integration

Configuration

```
trigger:
  type: event
  config:
    eventType: "com.company.order-completed"
    filterQuery: |
      event.type == "com.company.order-completed"
      AND order.total > 1000
```

Sending Business Events

Ingest events via API:

```
curl -X POST "https://&env&/api/v2/bizevents/ingest" \
-H "Authorization: Api-Token &token&" \
-H "Content-Type: application/json" \
-d '{
  "type": "com.company.order-completed",
  "data": {
    "order_id": "ORD-12345",
    "customer": "ACME Corp",
    "total": 1500.00
  }
}'
```

Event Data

Access event fields in tasks:

```
{{ event()["data"]["order_id"] }}
{{ event()["data"]["customer"] }}
{{ event()["data"]["total"] }}
```

Business events never open problems

That is a feature of this trigger, not a gap. A record ingested through

`/api/v2/bizevents/ingest` lands in Grail as a business event: queryable, and able to trigger a workflow exactly as shown above — but it never raises a Davis problem and never enters problem correlation. `dt.smartscape_source.id`, the correlation key that decides which Davis events merge into one problem (AIOPS-03 §1), is not part of this path and does not apply here.

If what you actually want is a problem, this is the wrong API. Custom *Davis* events go to `/api/v2/events/ingest` with an `entitySelector` naming the entity the signal is about. Omitting that selector is the classic mistake: the event associates with the environment entity rather than a real one, so it can never merge with anything and opens a brand-new problem on every firing. S2D-05 works through that failure in a scheduled-workflow context.

Sources: [Ingest an event — POST /api/v2/events/ingest \(DT docs\)](#) — "If not set, the event is associated with the environment (`dt.entity.environment`) entity.", [Avoid overalerting \(DT docs\)](#).

7. Trigger Data and Expressions

Accessing Trigger Data

Expression	Returns	Example
<code>{{ event() }}</code>	Full event object	<code>{"title": "...", ...}</code>
<code>{{ event()["field"] }}</code>	Specific field	<code>"High response time"</code>
<code>{{ trigger() }}</code>	Trigger metadata	<code>{"type": "davis-problem"}</code>
<code>{{ trigger().params }}</code>	On-demand params	<code>{"env": "prod"}</code>

Common Event Fields by Trigger Type

Detected Problem:

```
{{ event()["display_id"] }}      # P-12345
{{ event()["title"] }}          # Problem title
{{ event()["severity"] }}       # CRITICAL, HIGH, MEDIUM, LOW
{{ event()["status"] }}         # OPEN, RESOLVED
{{ event()["affected_entity_ids"] }} # Array of entity IDs
{{ event()["root_cause_entity_id"] }} # Root cause entity
{{ event()["management_zones"] }} # Array of MZ names (legacy – empty once
MZs are retired; do not route on this)
{{ event()["problem_url"] }}     # Link to problem
```

Detected Event (Metric):

```
{{ event()["metric_key"] }}     # Metric identifier
{{ event()["value"] }}          # Current value
{{ event()["threshold"] }}      # Threshold that was breached
{{ event()["entity_id"] }}      # Affected entity
```

Schedule:

```
{{ trigger()["scheduled_time"] }} # When scheduled to run
{{ trigger()["actual_time"] }}    # When actually started
```

8. Davis Problem Event Payload Reference

Verified against Dynatrace Workflows trigger reference (DT docs), May 2026.

Payload field names and lifecycle semantics can drift sprint-to-sprint — re-verify against your tenant version before building load-bearing routing logic on uncommon fields.

Workflows on a Detected Problem trigger receive a single `event` object representing the Davis problem at the moment the trigger fired. Sections 2 and 7 above show common access patterns; this section is the full reference: every field commonly seen in the payload, when it appears, how it joins to DQL, and which downstream notebooks consume it.

8.1. Top-Level Field Reference

Field	Type	Always present?	Notes
<code>event.kind</code>	string	Yes	Always <code>"DAVIS_PROBLEM"</code> for this trigger. Distinguishes from <code>DAVIS_EVENT</code> (raw signals).
<code>display_id</code>	string	Yes	Human-facing problem ID (<code>"P-12345"</code>). Use in notifications and ticket subjects.
<code>problem_id</code>	string	Yes	Internal problem ID. Use to join to fetch <code>dt.davis.problems</code> (see §8.3).
<code>event.id</code>	string	Yes	Event-record ID for this specific lifecycle update (different per OPEN/UPDATE/CLOSE firing).
<code>title</code> / <code>event.name</code>	string	Yes	Short problem title — <code>"High response time on checkout service"</code> . Both names appear; <code>event.name</code> is the DQL-canonical form.
<code>severity</code> / <code>event.category</code>	string	Yes	One of <code>AVAILABILITY</code> , <code>ERROR</code> , <code>SLOWDOWN</code> , <code>RESOURCE</code> , <code>CUSTOM_ALERT</code> , <code>MONITORING_UNAVAILABLE</code> , <code>INFO</code> . Older fields may also surface <code>CRITICAL</code> / <code>HIGH</code> / <code>MEDIUM</code> / <code>LOW</code> semantic levels.

Field	Type	Always present?	Notes
<code>status</code>	string	Yes	<code>ACTIVE</code> (open) or <code>CLOSED</code> . Older notebooks and docs sometimes show <code>OPEN</code> / <code>RESOLVED</code> — both names occur depending on surface.
<code>event.status_transition</code>	string	On UPDATE/CLOSE	<code>CREATED</code> , <code>UPDATED</code> , or <code>CLOSED</code> — tells the workflow <i>which lifecycle event</i> triggered it.
<code>start_time</code>	timestamp (ISO 8601)	Yes	When the problem opened.
<code>end_time</code>	timestamp (ISO 8601)	On CLOSE only	When the problem closed. Null/missing while active.
<code>affected_entity_ids</code>	string[]	Yes	All entities Davis correlated to this problem (services, hosts, processes, etc.).
<code>root_cause_entity_id</code>	string	When determined	Single entity Davis identified as root cause. May be empty/null on initial OPEN before causation analysis completes.
<code>impacted_entities</code>	object[]	Yes	Richer per-entity records — typically <code>{ id, name, type }</code> . Use when you need the entity <i>name</i> without an extra lookup.
<code>affected_entity_types</code>	string[]	Yes	Distinct entity types touched by this problem (<code>["SERVICE", "HOST"]</code>). Useful for routing without enumerating entity IDs.

Field	Type	Always present?	Notes
<code>management_zones</code>	<code>string[]</code>	Yes	Management-zone names this problem touches. Drives team routing in WFLOW-04.
<code>entity_tags</code>	<code>object[]</code>	When tagged	Tags on the affected entities (<code>[{"key": "team", "value": "checkout"}, ...]</code>). Custom-tag routing reads from here.
<code>problem_url</code>	<code>string</code>	Yes	Deep-link to the Davis Problems-app page for this problem. Always include in notifications.
<code>event.description</code>	<code>string</code>	Variable	Longer human-readable description; not always populated. Treat as optional.

Sources: [Workflow triggers \(DT docs\)](#), [Workflow reference / Jinja expressions \(DT docs\)](#), [Davis Problems app \(DT docs\)](#). **Derived:** the OPEN vs UPDATE vs CLOSE field-presence column synthesizes the trigger reference with observed payloads — `end_time` and `event.status_transition` are the load-bearing differentiators across lifecycle stages.

Where the entity fields come from. `affected_entity_ids`, `root_cause_entity_id`, and `impacted_entities` are populated from the Davis events Dynatrace grouped into this problem, and that grouping runs on `dt.smartscape_source.id` — the Smartscape entity ID each event carries. Events naming the same entity within the correlation window merge into the single problem this payload describes.

That has a direct consequence for anyone reading these fields in a workflow: a problem assembled from **unattributed** events arrives with those fields thin or empty, so routing that branches on `affected_entity_types`, `entity_tags`, or `root_cause_entity_id` silently falls through to its default. If a workflow keeps landing on the fallback branch, suspect the upstream detector's event template before rewriting the routing logic — an event template that leaves `dt.smartscape_source.id` unset also opens a separate problem on every firing, so the symptom usually arrives alongside a problem-count spike. AIOPS-03 §1

covers the correlation rules, AIOPS-02 §8 finds unattributed detectors in your own tenant, and ALERT-99 §3 explains which Davis data object to count when you audit.

Sources: [Avoid overalerting \(DT docs\)](#).

8.2. Lifecycle Field Presence

A single workflow can fire on problem **CREATED**, **UPDATED**, and **CLOSED**. The payload shape differs at each stage — guard logic that assumes all fields are populated will silently mis-route close events.

Field	CREATED	UPDATED	CLOSED
<code>display_id</code> , <code>problem_id</code> , <code>title</code>	✓	✓	✓
<code>severity</code> , <code>status</code>	✓	✓	✓ (<code>CLOSED</code>)
<code>event.status_transition</code>	<code>CREATED</code>	<code>UPDATED</code>	<code>CLOSED</code>
<code>start_time</code>	✓	✓	✓
<code>end_time</code>	—	—	✓
<code>root_cause_entity_id</code>	sometimes empty	usually populated	populated
<code>affected_entity_ids</code>	initial set	may expand	final set
<code>entity_tags</code>	from initial entities	may expand	final set
<code>problem_url</code>	✓	✓	✓

In community practice the safest routing pattern is: branch on `event.status_transition` first (and fall back to `status` if absent), then read only the fields guaranteed for that branch.

8.3. Joining the Payload to DQL

The workflow payload is a snapshot. For longer-window analysis (problem history, MTTR, fleet-wide patterns), join the payload's `problem_id` to the canonical Grail table:

```
// Hydrate the workflow payload against the full Davis problems record.
// Substitute {{ event()["problem_id"] }} via a DQL workflow task.
fetch dt.davis.problems, from:-7d
| filter event.id == "{{ event()['problem_id'] }}"
| fields display_id,
        event.name,
        event.category,
        event.status,
        event.start,
        event.end,
        affected_entity_ids,
        root_cause_entity_id,
        management_zones
| limit 1
```

Why `dt.davis.problems` and not `dt.davis.events` : `dt.davis.problems` is the canonical problems data object; `dt.davis.events` carries only `DAVIS_EVENT` records (raw signals that *feed* problem detection, not the problems themselves). Querying `dt.davis.events` for problem records returns zero rows on modern tenants.

For aggregate views (e.g., "all problems for the same entity this week") swap the filter:

```
fetch dt.davis.problems, from:-7d
| filter in("{{ event()['affected_entity_ids'][0] }}", affected_entity_ids)
| summarize problem_count = count(), by:{event.category, event.status}
| sort problem_count desc
```


8.4. Common Access Patterns

```
# Display ID and link – every notification
{{ event()["display_id"] }}
{{ event()["problem_url"] }}

# Severity-based routing (see WFLOW-04 §3 routing-by-severity)
{% if event()["severity"] == "AVAILABILITY" %} page on-call
{% elif event()["severity"] == "ERROR" %}      notify channel
{% else %}                                     log only
{% endif %}

# Lifecycle branch – distinguishes open/update/close in the same workflow
{% if event()["event.status_transition"] == "CLOSED" %} close ticket
{% else %}                                           update ticket
{% endif %}

# Custom-tag access (entity_tags is an array of {key, value} objects)
{% for tag in event()["entity_tags"] %}
  {% if tag["key"] == "team" %}{{ tag["value"] }}{% endif %}
{% endfor %}

# Entity-type filtering – route by what kind of entity is involved
{% if "SERVICE" in event()["affected_entity_types"] %} service-team channel
{% elif "HOST" in event()["affected_entity_types"] %}   infra channel
{% endif %}

# Root cause name (no extra lookup needed – impacted_entities carries names)
{% for ent in event()["impacted_entities"] %}
  {% if ent["id"] == event()["root_cause_entity_id"] %}{{ ent["name"] }}{%
endif %}
{% endfor %}
```

8.5. Cross-References

- **WFLOW-04 §3 Routing by Severity** consumes `severity` and `management_zones` from this payload to fan notifications out to team channels.
- **WFLOW-04 §4 Routing by Team/Service** reads `entity_tags` (custom tags like `team:checkout`) and `affected_entity_types`.
- **WFLOW-05 Incident Management** uses `display_id`, `problem_url`, `status`, and `start_time` to create and update tickets through the lifecycle.
- **WFLOW-07 Remediation** branches on `root_cause_entity_id` and `title` to pick the right runbook task, and joins `problem_id` to `dt.davis.problems` for richer context

before acting.

8.6. Honest Caveats

- **Field-name drift:** Some fields surface under two names (`title` ↔ `event.name` , `severity` ↔ `event.category` , `status` value `OPEN` vs `ACTIVE`). Older workflow templates and docs frequently use the older form; new workflows should prefer the `event.*` canonical names because they match the DQL field surface in `dt.davis.problems` .
- **status value vocabulary:** Modern Davis records use `ACTIVE` / `CLOSED` ; some legacy notebooks (including earlier sections of this notebook) still show `OPEN` / `RESOLVED` . Both can appear in payloads depending on tenant version — guard with a set check rather than `==` .
- **Re-verify before load-bearing logic:** This reference is dated 05/21/2026. The Workflows trigger schema is product surface and can change in a sprint. Spot-check a real payload (use a logging task or send-to-Slack-as-debug task) in your tenant before relying on field availability in production routing.

Query Detected Problems

```
// Recent detected problems that could trigger workflows
fetch events, from: now() - 24h
| filter event.kind == "DAVIS_PROBLEM"
| fields timestamp,
          display_id,
          event.name,
          severity,
          status,
          affected_entity_ids,
          root_cause_entity_id
| sort timestamp desc
| limit 20
```

```
// Problem count by severity (last 7 days)
fetch events, from: now() - 7d
| filter event.kind == "DAVIS_PROBLEM"
| filter status == "OPEN"
| summarize problem_count = count(), by:{severity}
| sort problem_count desc
```

```
// Business events that could trigger workflows
fetch bizevents, from: now() - 24h
| summarize event_count = count(), by:{event.type}
| sort event_count desc
| limit 20
```

Next Steps

Now that you understand triggers, learn to send notifications:

Recommended Path

1. **WFLOW-03: Alert Notification Basics** - Slack, Teams, email notifications
2. **WFLOW-04: Advanced Notification Routing** - Conditional routing
3. **WFLOW-07: Problem-Triggered Remediation** - Auto-remediation patterns

Key Takeaways

- **Detected Problem** triggers for AI-detected incidents
- **Detected Event** triggers for metric thresholds
- **Schedule** triggers for recurring tasks
- **On-Demand** triggers for testing and API integration
- **Event** triggers for business events
- Use expressions like `{{ event()["field"] }}` to access data

Summary

In this notebook, you learned:

- All five trigger types and when to use each
- How to configure Detected Problem triggers with filters
- How to set metric thresholds with Detected Event triggers
- Cron expressions for schedule triggers
- On-demand execution via UI and API

- [Business event triggers for custom automation](#)
 - [How to access trigger data in expressions](#)
-

References

- [Workflow triggers \(DT docs\)](#)
 - [Workflow reference / Jinja expressions \(DT docs\)](#)
 - [Davis Problems app \(DT docs\)](#)
 - [Business Observability umbrella \(DT docs\)](#)
 - [Workflows umbrella \(DT docs\)](#)
 - [Cron expression sandbox \(crontab.guru\)](#)
 - [Upgrade guide — alerting and notifications \(DT docs\)](#) — old → new mapping; states the Management Zone filter is no longer supported
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.